Studio Nx

Backend(FastAPI/Python) 04 — unexpected_error_handler()

2026-10-12

Backend(FastAPI/Python) 04 — `unexpected_error_handler()`

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-03 백엔드 공통: 오류 응답 형식 (처리기 네 개 중 마지막)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	03 `http_error_handler()` · 05 `decode_access_token()` (토큰 검사)

1. 하는 일

01~03이 맡지 않은 **모든 오류**(버그, DB 연결 끊김처럼 예상하지 못한 오류)를 500 응답으로 바꿉니다. 응답에는 일반적인 문장만 넣고, 오류 내용은 넣지 않습니다.

```
500  {"error": {"code": "INTERNAL_ERROR", "message": "Something went wrong. Please try again.", "detail
s": null}}
```

2. 언제 쓰나

코드가 예상하지 못한 이유로 멈췄을 때입니다. 정상이라면 일어나지 않아야 하는 오류입니다.

상황	원래 오류 (서버 안)	응답
코드 버그 (`None`의 칸을 읽는 등)	`AttributeError: 'NoneType' object has no attribute 'email'`	500, 일반 문장
DB 서버가 꺼짐	`OperationalError: (2003, "Can't connect to MySQL server on 'localhost'")`	500, 일반 문장
파일 저장 실패	`PermissionError: [Errno 13] … C:\Users\…\uploads\…`	500, 일반 문장

오류 내용이 남는 곳

응답에는 없지만 오류 내용이 **사라지지는 않습니다.** 이 처리기가 응답을 보낸 뒤 Starlette가 원래 오류를 다시 던지고, 두 곳이 그것을 기록합니다.

순서	일어나는 일
① 라우터에서 오류	`RuntimeError(...)`가 납니다
② 이 처리기	사용자에게 500과 일반 문장으로 응답합니다
③ Starlette	응답을 보낸 뒤 **원래 오류를 다시 던집니다** (`raise exc`)
④ uvicorn	서버를 실행한 터미널에 오류 전체(어느 파일의 몇째 줄인지)를 출력합니다
⑤ Sentry	`SENTRY_DSN`이 설정되어 있으면 Sentry로 보냅니다 (`core/sentry.py`가 비밀번호, 토큰, 쿠키를 지운 뒤에 보냅니다)

보는 사람	보는 것
사용자 (화면)	"Something went wrong. Please try again."

보는 사람	보는 것
개발자	터미널 로그와 Sentry에서 오류 종류, 내용, 위치 전체를 봅니다

처리기 네 개 정리 (P1-03 오류 응답 형식 완료)

#	처리기	맡는 오류	상태
01	`app_error_handler`	이 프로젝트의 코드가 일부러 던진 `AppError`	오류에 적힌 값
02	`validation_error_handler`	입력 형식 오류	422
03	`http_error_handler`	없는 주소, 다른 방식 등	404, 405 …
04	**`unexpected_error_handler`**	그 밖의 모든 오류	500

3. 코드

import (파일 맨 위)

```
from enum import StrEnum

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException

from app.schemas.common import ErrorBody, ErrorResponse
```

`errors.py` 맨 위의 import입니다. 이 함수는 **`Request`와 `JSONResponse`**만 사용합니다. `Exception`은 파이썬 기본이라 import할 필요가 없습니다.

함수

```
async def unexpected_error_handler(request: Request, exc: Exception) -> JSONResponse:
    return error_json(500, ErrorCode.INTERNAL_ERROR, "Something went wrong. Please try again.")
```

`register_exception_handlers()`에 등록합니다.

```
app.add_exception_handler(Exception, unexpected_error_handler)
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다 (예: `Request`)

식	뜻
`exc: Exception`	`Exception`은 거의 모든 오류의 부모입니다. 그래서 01~03이 맡지 않은 오류가 모두 여기로 옵니다
`exc`를 쓰지 않음	일부러 그렇게 했습니다. 오류 내용을 응답에 넣지 않으므로 받기만 합니다
`add_exception_handler(Exception, …)`	FastAPI는 `Exception` 처리기를 다른 처리기와 따로, **가장 바깥**(`ServerErrorMiddleware`)에 둡니다. 더 구체적인 처리기(01~03)가 먼저 잡고, 아무도 잡지 않은 오류만 여기로 옵니다

4. 흐름

1. 01~03이 잡지 않은 오류가 가장 바깥까지 올라옵니다.
2. 이 처리기가 500과 일반 문장으로 응답을 만듭니다.
3. Starlette가 응답을 보낸 뒤 원래 오류를 다시 던지고, 터미널 로그와 Sentry가 그것을 기록합니다.

5. 왜 이렇게

선택	이유
오류 내용을 응답에 넣지 않음	오류 문장에는 SQL, 파일 경로, DB 주소, 비밀값이 섞일 수 있습니다 (§2의 예). 그대로 보내면 공격자에게 서버 구조를 알려 주게 됩니다 (PRD 01 §5)
기록하는 코드를 따로 쓰지 않음	Starlette가 오류를 다시 던져서 uvicorn과 Sentry가 이미 기록합니다. 여기서 또 기록하면 Sentry에 같은 오류가 두 번 쌓입니다
응답 모양을 다른 오류와 같게 맞춤	`api.js`가 `INTERNAL_ERROR`로 읽을 수 있습니다. 기본 500 응답은 JSON이 아니라 `Internal Server Error`라는 글자라서, 프론트가 형식을 알 수 없습니다
문구는 "다시 시도"	DB 연결이 잠깐 끊긴 경우처럼 다시 하면 되는 일이 많습니다. 오류가 계속되면 Sentry에서 알 수 있습니다

6. 시험 (`tests/test_errors.py`에 2개 추가)

시험용 주소가 `RuntimeError("SELECT * FROM users WHERE password_hash='…'")`를 던지게 했습니다.

경우	결과
응답 확인 (`raise_server_exceptions=False`)	500이고, 본문이 정확히 위의 JSON이며, 응답에 `password_hash`와 `RuntimeError`가 없습니다
다시 던지는지 확인 (기본 설정)	시험 도구에서 `RuntimeError`가 그대로 올라옵니다. 그래서 로그와 Sentry가 기록할 수 있습니다

`raise_server_exceptions=False`: 시험 도구(`TestClient`)는 기본적으로 서버 오류를 다시 던져서 시험을 멈춥니다. 실제 응답을 보려고 이 옵션을 껐습니다.

전체 시험 26개가 통과했고, `ruff` 검사도 통과했습니다.